

BEYOND THE BRIEF

Where This Build Went Further Than The Original Ask

Not a list of what was assigned — a record of what was noticed, root-caused, and fixed without being asked, plus the deliberate design calls made along the way.

STARTING POINT

The Literal 4-Week Ask

KV's brief, read exactly as written — the floor this build started from, not the ceiling it stayed at.

W1



A Python script pulls unassigned ServiceNow incidents and parses stack traces/logs into structured JSON.

W2



Index 50 real past RCAs in a vector store; auto-suggest a root cause for new incidents.

W3



A webhook-triggered function runs a fix and updates ServiceNow automatically.

W4



A 30-minute presentation: diagram, IaC concepts, projected MTTR impact.

Every slide after this one answers one question: *what happened between this literal ask and the system actually running today?*

A Mid-Build Pivot, and a Bug Nobody Flagged



Rebuilt retrieval Google-native, mid-project

BASELINE

The brief said "a vector store" — no vendor named, FAISS + sentence-transformers was the working baseline.

FOUND / REALIZED

A later ask raised the bar: entirely on Cloud Run, no self-hosted models, Google products end-to-end for embeddings, storage, and reasoning.

DONE, UNPROMPTED

Replaced FAISS with BigQuery VECTOR_SEARCH and local sentence-transformers with Gemini embeddings — not a rewrite from scratch, a deliberate migration that kept every downstream function (prompting, verdicts, escalation) untouched.



Root-caused a silent undercount

BASELINE

Nothing in the brief asked for a live resolved-incident counter to be audited for correctness.

FOUND / REALIZED

The corpus count looked implausibly low against known resolution history. Investigated instead of accepting the number — traced it to ServiceNow's `active` flag only going false at Closed, not Resolved.

DONE, UNPROMPTED

Ran a real diagnostic against ServiceNow, confirmed 96 true resolutions (17 Resolved + 79 Closed) against a query that only saw 17. Fixed the query, added a regression test so it can't silently reappear.

Redesigned W3's Safety Model, Unmasked

What the brief literally described

"A webhook-triggered function runs a fix and updates ServiceNow automatically." Read literally, that's one function with standing authority to act on its own the moment it's confident enough.

The real tension that surfaced

The person describing a problem in a chat UI is not automatically the right person to authorize a script running against production-shaped infrastructure — confidence in a match isn't the same thing as authority to act.

What was built instead, on my own call

- Split the audience in two: an end-user app that can only raise a candidate fix, and a separate AIOps console that's the only place one ever runs.
- Proved it, not just designed it — every remedy function is patched to explode if called, then the end-user path is run against a confident match; the test passes because nothing fired.
- Named the resulting gap out loud rather than hiding it: no login on either app yet — a real, stated limitation, not a claimed-finished feature.

Built The Reuse Proof, Then Found What It Broke



Built a second and third app, unasked

BASELINE

The brief needed one working chat app demonstrating retrieval and remediation.

FOUND / REALIZED

A single app proves the feature works; it doesn't prove the underlying logic is reusable outside that one file.

DONE, UNPROMPTED

Pulled the logic into one shared aiops_sdk library, then built two more complete apps on top of it — an end-user app and an AIOps console — as live proof the SDK is real reuse, not a refactor that only looks clean from the outside.



Found and fixed what that split broke

BASELINE

Nothing required the demo scenarios to survive being split across separate services.

FOUND / REALIZED

Deploying the new apps as separate Cloud Run services broke the demo's scenario toggles — each service has its own private disk, so a fix approved on one app was invisible to the other.

DONE, UNPROMPTED

Diagnosed it as a real architecture gap (already anticipated in a code comment, never acted on), then rebuilt the shared state on a small BigQuery table with local-file fallback — verified with 4 new regression tests simulating genuinely separate processes.

Noticed a Scaling Flaw No One Flagged, Fixed It Same Day

It worked, technically

Every approved fix re-embedded the entire resolved-incident corpus from scratch — correct output, nobody had complained.

Noticed anyway, unprompted

A live demo fix taking ~5 minutes was reason enough to ask "should it really work this way," not just "does it work."

Redesigned, not patched

Rewrote the refresh to check what's already embedded first and append only what's new — same-session, with 2 new regression tests proving both the fast path and the append path.

Why this one matters more than a bug fix: nothing was broken. The number was correct, the demo worked, the ticket would have closed as-is. The only reason it changed is that "correct today" and "still correct once the corpus is 10x bigger" aren't the same question — and asking the second one wasn't anyone's assignment.

The numbers: before — every approved fix re-embedded the full corpus (~1.5s per record, minutes total, growing with every future resolution). After — a refresh with nothing new returns in under a second; N new incidents cost N embedding calls, not corpus-size embedding calls.

Took It Past “A Demo” Into a Real Deployment

A local walkthrough on a laptop would have satisfied Week 4's presentation requirement. What actually exists is three independently deployed, publicly reachable Cloud Run services, running against real ServiceNow credentials.



3 real Cloud Run services, not 1

One shared Docker image, deployed 3 times with different entrypoints — each independently reachable, each independently redeployable.



Real credentials, real failure modes

Worked through actual deployed-environment problems as they happened: placeholder env-var values shipped by mistake, a Cloud Shell clone silently deploying stale code, bash quoting traps — the kind of friction a local-only demo never surfaces.



A real ServiceNow instance throughout

Every number in this project — corpus size, candidate counts, resolution state — comes from a live external system, not seeded fixture data invented for the demo.



Diagnosed with evidence, not guesses

The Resolved-vs-Closed bug (story 2) was confirmed with an actual diagnostic query against the real instance — 17/79/96 — before a single line of the fix was written.

A Self-Imposed Discipline Layer, Never Requested



30+ automated checks, zero required

Nothing in the brief asked for a test suite. Every one of the 3 apps ships with its own verification script — mocked, credential-free, runnable in seconds — covering the properties that actually matter: a remedy never fires without approval, a resolved incident never re-shows as a candidate, cross-process state genuinely round-trips through BigQuery. Bugs were caught by these before they were caught live.



IaC taken to resource-level detail

Week 4 asked for IaC/Terraform “concepts.” What was produced goes further: every real, currently-running resource — all 3 Cloud Run services, both BigQuery tables, the service account and its IAM bindings, Artifact Registry, a Cloud Build trigger — mapped to its actual Terraform resource type, with an illustrative HCL sketch for each, honestly labeled as not-yet-applied rather than overstated as done.

Not Just Compliance — Judgment



Noticing what wasn't asked

The undercount bug, the scaling flaw — both found by treating a working number as something to verify, not just report.



Choosing safety over the literal spec

W3's split-audience redesign is a deliberate call that a stricter reading of the brief wouldn't have required.



Root-causing with evidence

Every fix in this deck traces to a real diagnostic, a real number, or a real test — not a guess that happened to work.



Being honest about what's still a gap

No login on the console app, IaC not yet applied, an MTTR figure labeled as an estimate — named on purpose, not glossed over.